Clustering

Clustering

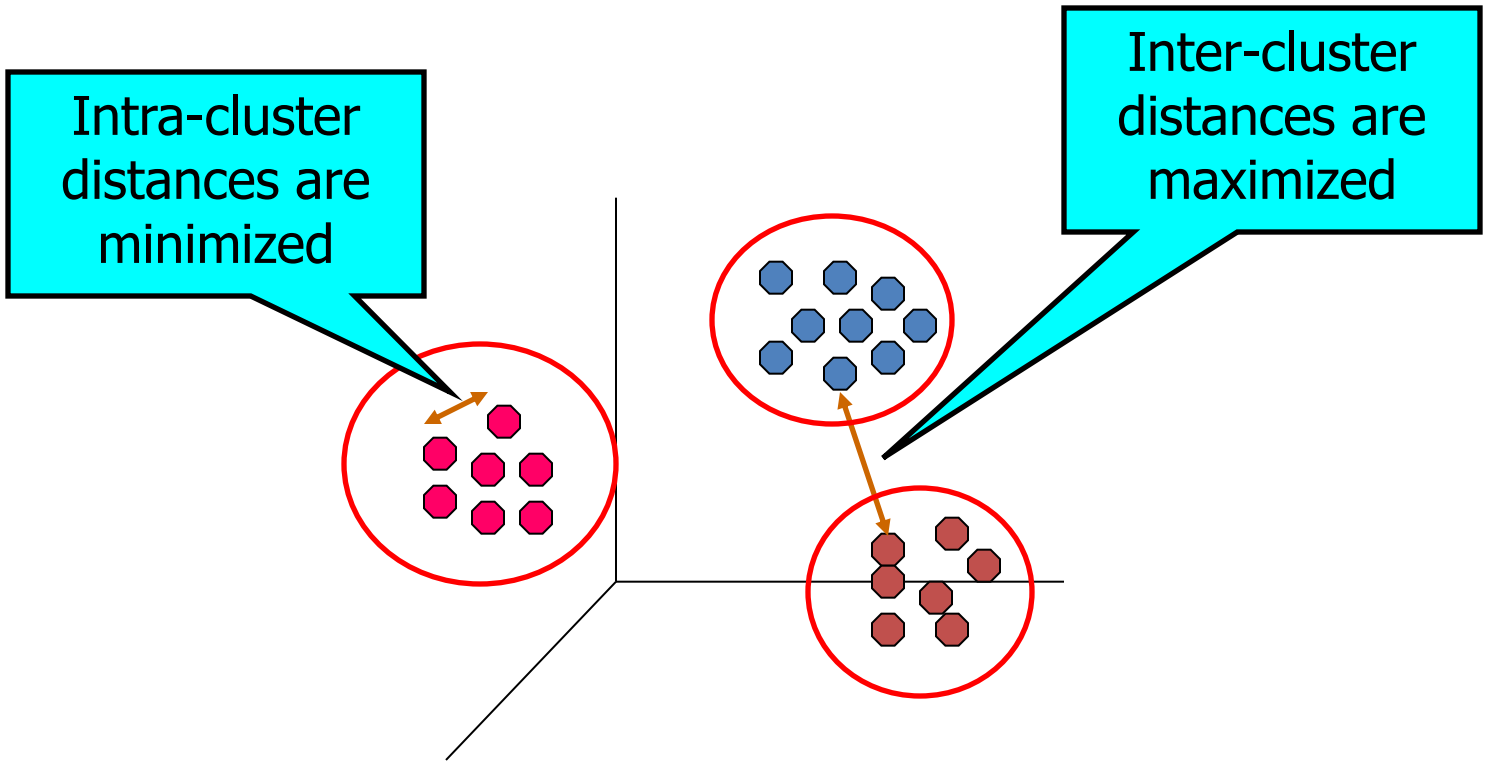
Clustering is a data mining technique which groups unlabeled data based on their similarities or differences. Clustering algorithms are used to process raw, unclassified data objects into groups represented by structures or patterns in the information.

Clustering algorithms can be categorized into a few types, specifically exclusive, overlapping, hierarchical, and probabilistic.

What is Cluster Analysis?

Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups

Deciding the distance/similarity metric is crucial



Notion of a Cluster can be Ambiguous

Poll: How many clusters do you see?

Six Clusters

Two Clusters

Four Clusters

Types of Clusterings

A clustering is a set of clusters

Important distinction between hierarchical and partitional sets of clusters

Partitional Clustering

Dividing data objects into non-overlapping subsets (clusters) such that each data object is in exactly one subset

Overlapping means that **a single data point can belong to more than one cluster** at the same time.

Hierarchical clustering

A set of nested clusters organized as a hierarchical tree

Partitional Clustering

Definition:
Partitional clustering divides a dataset into non-overlapping clusters such that each data point belongs to exactly one cluster. The most common example is K-Means clustering.

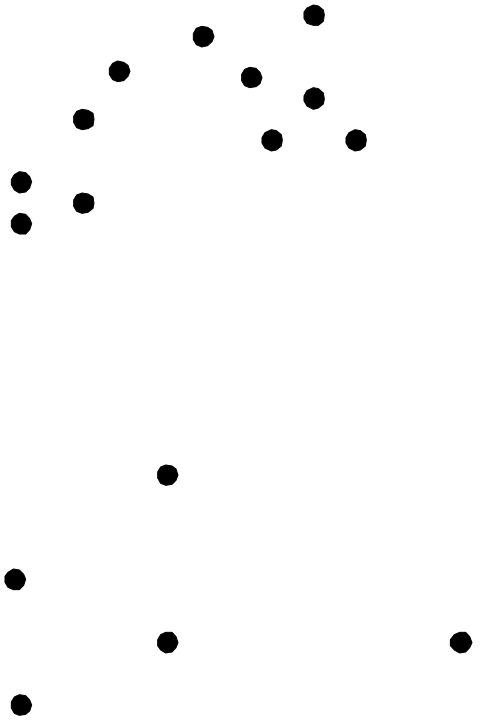
Key Characteristics:

- Flat structure: No hierarchy; each cluster is independent.
- Exclusive assignment: Each data point is assigned to only one cluster.
- Predefined number of clusters: The number of clusters (k) is often specified beforehand.
- Optimization-based: Algorithms try to optimize an objective function, e.g., minimize intra-cluster variance.

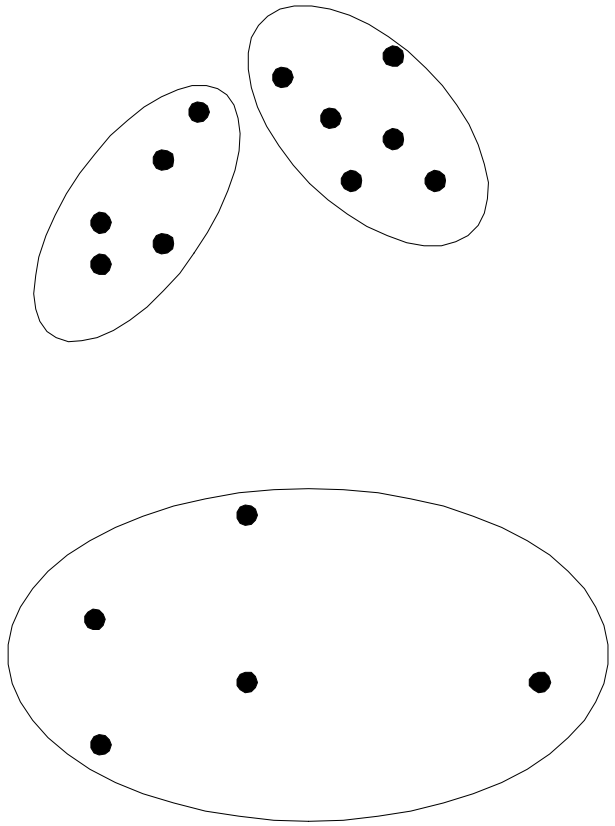
Example Algorithms:

- K-Means
- K-Medoids
- DBSCAN (though it allows noise points)

Partitional Clustering



Original Points



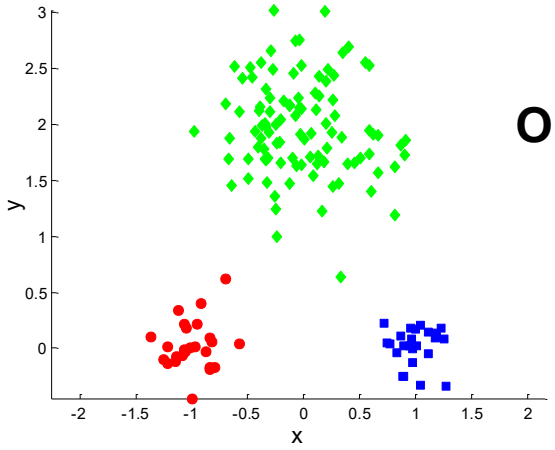
A Partitional Clustering

What is K-Means Clustering?

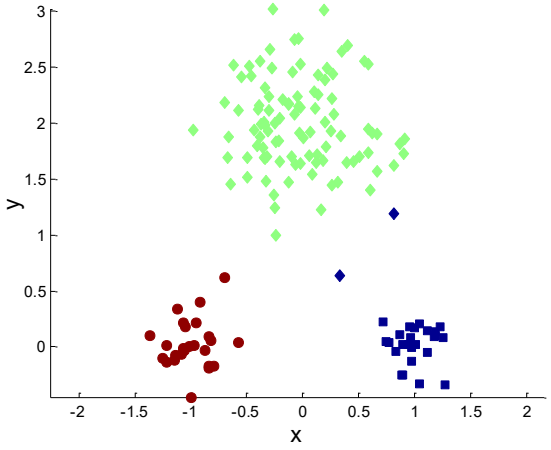
K-Means is a partitional clustering algorithm that tries to divide data into k clusters by:

- Initializing k centroids (usually randomly).
- Assigning each data point to the nearest centroid.
- Recalculating centroids.
- Repeating steps 2–3 until convergence (e.g., centroids stop moving).

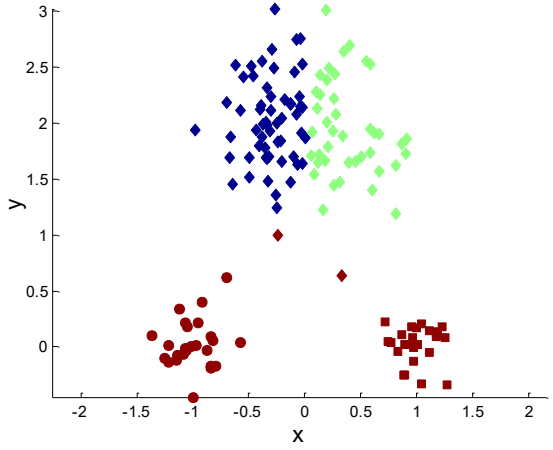
Two different K-means Clusterings



Original Points

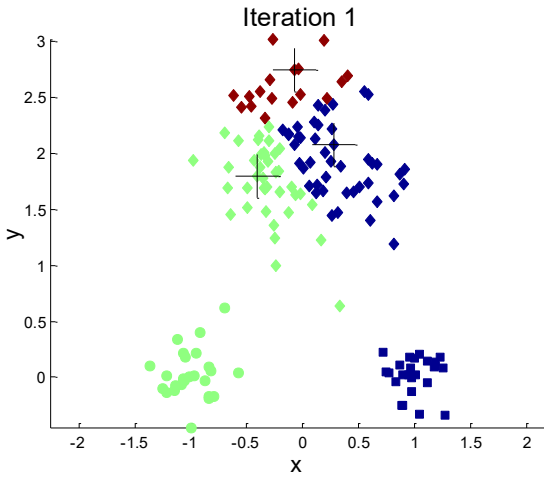


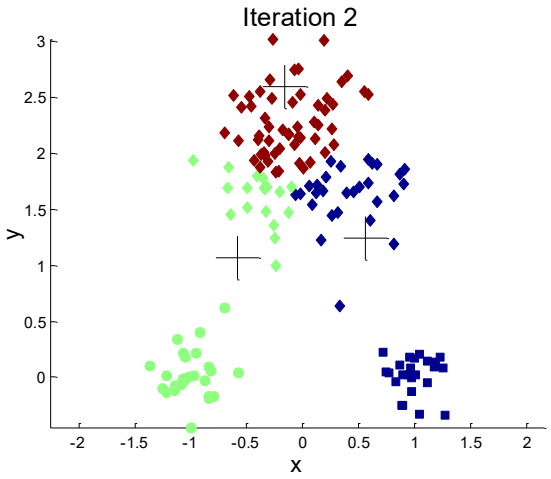
Optimal Clustering

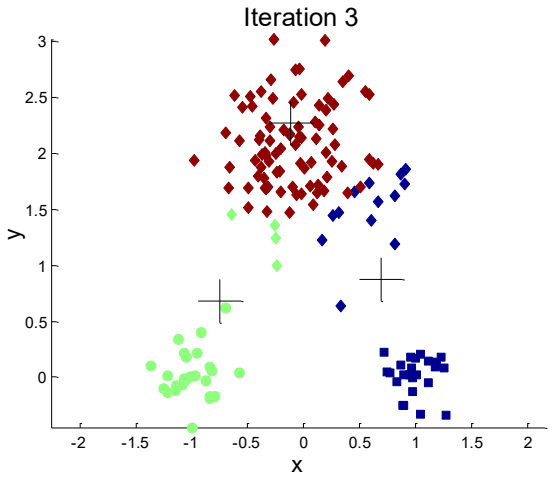


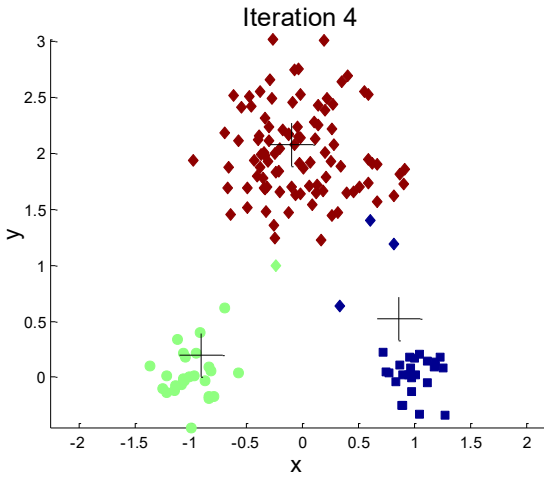
Sub-optimal Clustering

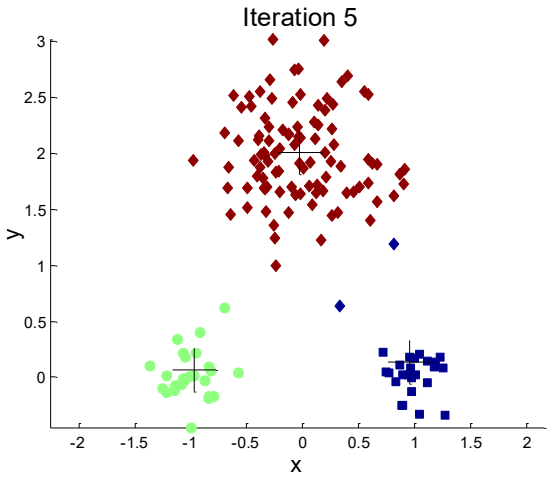
Importance of Choosing Initial Centroids (1)

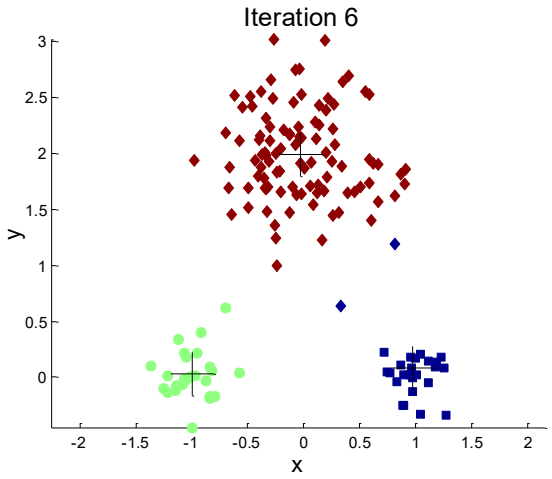












K-Means groups data into clusters by minimizing distance to centroids.

Each iteration updates cluster assignments and centroids.

🎯 1. Centroid Calculation Equation

The centroid is the **mean (average) of all points in a cluster**.

$$c_j = \frac{1}{N_j} \sum_{i \in C_j} x_i$$

Where:

- c_j = centroid of cluster j
- N_j = number of points in cluster j
- x_i = data points in the cluster
- C_j = set of points belonging to cluster j

Example Data

✅ Example

Points in Cluster 1:

$$(2, 4), (4, 6), (6, 8)$$

Centroid:

$$c_1 = \left(\frac{2+4+6}{3}, \frac{4+6+8}{3} \right)$$

$$c_1 = (4, 6)$$

K-Mean Equation

🎯 2. K-Means Objective Function

K-Means tries to minimize the total distance between points and centroids.

$$J = \sum_{j=1}^{k} \sum_{i \in C_j} ||x_i - c_j||^2$$

Meaning:

- J = clustering error (cost function)
- k = number of clusters
- x_i = data point
- c_j = centroid
- $||x_i - c_j||^2$ = squared Euclidean distance

👉 K-Means keeps updating centroids until this error becomes as small as possible.

Distance Equation

🎯 Distance Equation (used to assign clusters)

Usually Euclidean distance:

$$d(x, c) = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$$

👉 Each point goes to the **nearest centroid**.

K-Means Process Summary

Step	Equation Used
Assign points to nearest cluster	Distance equation
Update centroid	Mean equation
Optimize clusters	Objective function

Process

- **“K-Means works by repeatedly assigning points to the nearest center, then recalculating the center as the average of all points in that cluster.”**

🎯 Real Example (Student Performance)

Assume we have students based on:

Student	Math	Programming
A	90	85
B	88	80
C	30	35
D	25	30
E	60	65

Step-by-Step

Step 1: Each point is its own cluster

{A}, {B}, {C}, {D}, {E}

Step 2: Compute distances (Euclidean)

Example:

$$d(A,B) = \sqrt{(90-88)^2 + (85-80)^2} = \sqrt{4+25} = \sqrt{29}$$

👉 A and B are very close

Step 3: Merge closest clusters

- Merge → **{A, B}**
- Merge → **{C, D}**

Now:

```
{A,B}, {C,D}, {E}
```

Step 4: Continue merging

- E is closer to (A,B) than (C,D)

```
{A,B,E}, {C,D}
```

🌳 Dendrogram Interpretation

- Height = distance between clusters
- Cut the tree → choose number of clusters

👉 Example:

- Cut early → 3 clusters
- Cut later → 2 clusters

The figure shows K-Means clustering over several iterations.

At the beginning, the algorithm chooses initial centroids. If these centroids are not well placed, the clustering may take more iterations or may give a poor final result.

In each iteration:

Iteration 1:
The points are assigned to the nearest centroid. Some clusters are still mixed because the centroids are not yet in the best position.

Iteration 2:
The centroids move toward the average position of the points assigned to them. The clusters start becoming clearer.

Iteration 3:
More points are reassigned correctly. The centroids continue moving.

Iteration 4:
The clustering becomes more stable. Most points now belong to the correct groups.

Iteration 5:
The centroids stop changing significantly. This means K-Means has converged.

The main idea: choosing good initial centroids is important because bad initialization can lead to slow convergence or incorrect clusters. This is why we often use K-Means++, which chooses better starting centroids.

✅ Step-by-step K-Means (5 Iterations)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load dataset
iris = load_iris()
X = iris.data[:, :2]  # use only 2 features for visualization

# Scale data
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Number of clusters
k = 3

# Random initial centroids (like bad initialization)
np.random.seed(42)
centroids = X[np.random.choice(len(X), k, replace=False)]

def assign_clusters(X, centroids):
```

```
 distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
    return np.argmin(distances, axis=1)

def update_centroids(X, labels, k):
    return np.array([X[labels == i].mean(axis=0) for i in range(k)])

# Run 5 iterations
for i in range(5):
    labels = assign_clusters(X, centroids)

    plt.figure()
    plt.scatter(X[:, 0], X[:, 1], c=labels)
    plt.scatter(centroids[:, 0], centroids[:, 1], s=200, marker='X')
    plt.title(f"Iteration {i+1}")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()

    # Update centroids
    centroids = update_centroids(X, labels, k)
```

🔍 What this code shows

Each loop represents:

- Assign clusters → each point goes to nearest centroid
- Plot current clusters + centroids
- Update centroids → move to the mean
- Repeat

🎯 **Important insight**

Iteration 1: messy clusters (bad initial centroids)

Iteration 2–3: centroids move → clusters improve

Iteration 4–5: stabilizes (convergence)

Hierarchical Clustering

Definition:
Hierarchical clustering builds a tree (dendrogram) of nested clusters by either:

Agglomerative approach (bottom-up): Start with each point as its own cluster and merge them step by step.

Divisive approach (top-down): Start with one large cluster and divide it recursively.

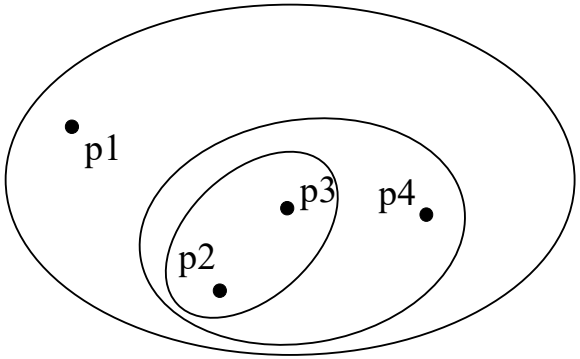
Key Characteristics:

- Nested structure: Clusters are organized as a tree of nested groups.
- No need to specify k: You can choose the level of granularity by cutting the dendrogram.
- Flexible: Can represent multilevel groupings of data.

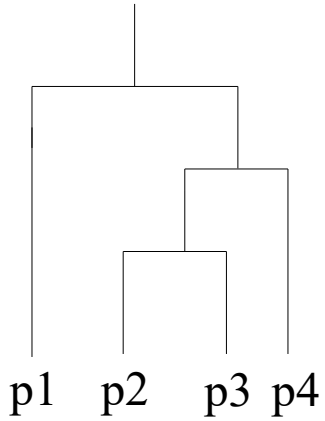
Example Algorithms:

- Agglomerative Hierarchical Clustering
- DIANA (Divisive Analysis)

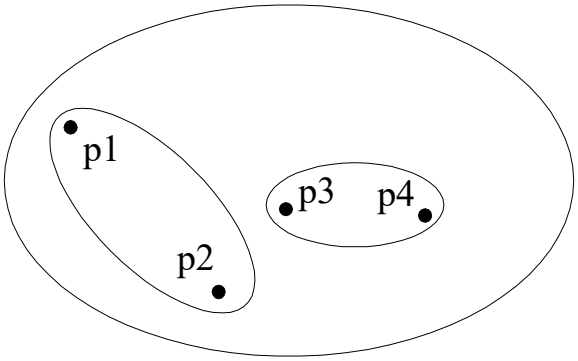
Hierarchical Clustering



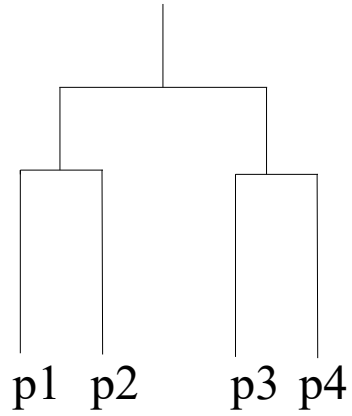
Traditional Hierarchical Clustering



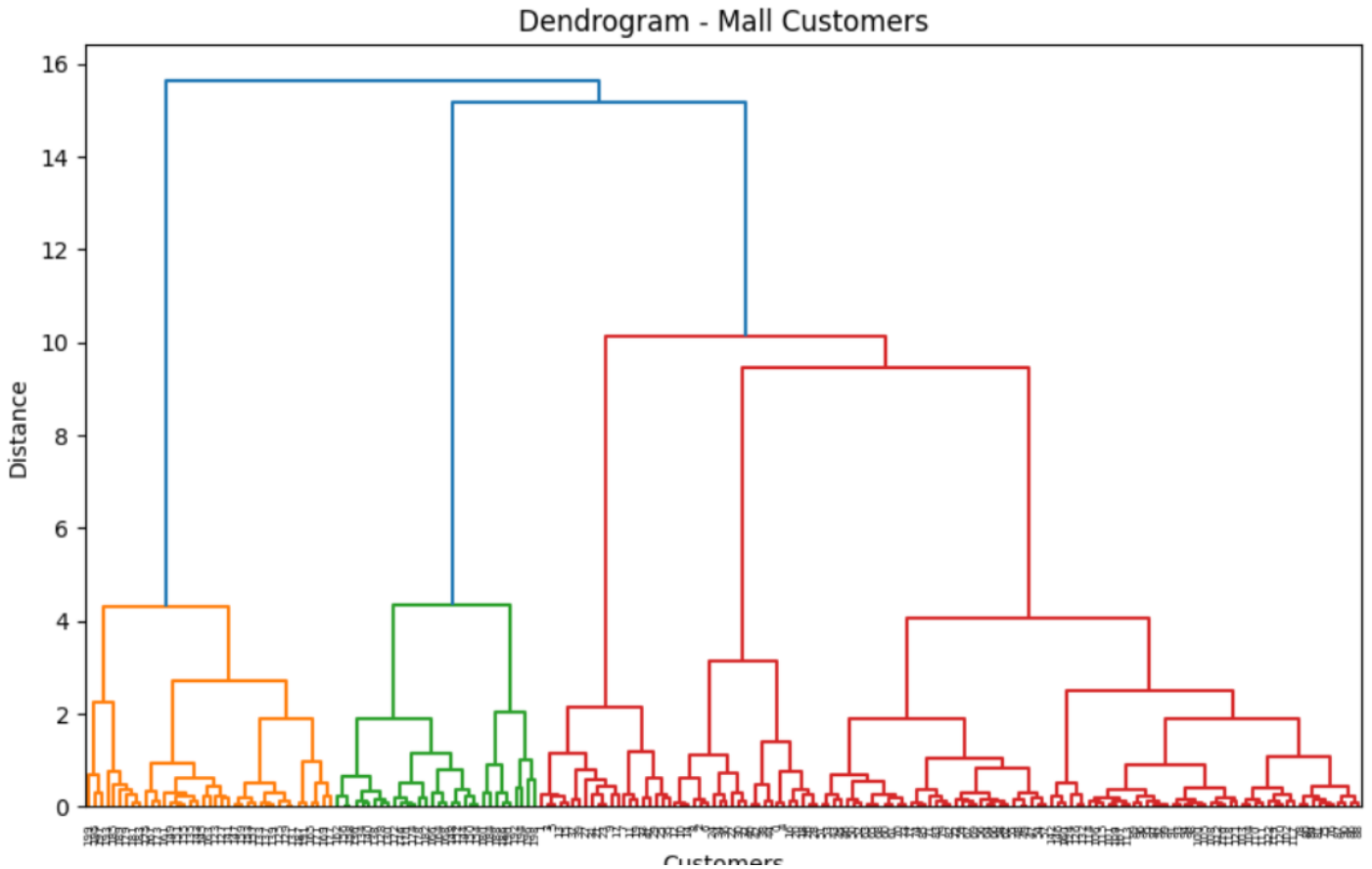
Traditional Dendrogram



Non-traditional Hierarchical Clustering



Non-traditional Dendrogram


Dendrogram - Mall Customers
Distance
16
14
12
10
8
6
4
2
0
Customers

Interpretation (Very Important for Teaching)

You will typically see clusters like:

Cluster	Meaning
High income + High spending	Premium customers 💎
High income + Low spending	Careful customers
Low income + High spending	Impulsive buyers
Low income + Low spending	Budget customers
Average	Average group

Final Hierarchy Summary

Level	Distance	Clusters	Meaning
0	0	200	Individual customers
1	0–2	~150	Very similar pairs
2	2–5	~50	Small groups
3	5–10	~3–5	Meaningful segments ✅
4	10–15	2	Broad groups
5	15+	1	All merged